

Component Developer Guide

Contents

Day-to-Day Workflow	1
Create a feature branch	1
Make changes and commit	1
Keep your branch up to date	2
Push and create a Pull Request	2
After your PR merges	3
Best Practices	3
Branching conventions	3
Commit messages	3
What NOT to commit	4
Code review etiquette	4
IP model conventions	5
Working with Models	6
Standalone model development	6
VP-integrated development	6
Cross-model changes	6
Documentation and PDF export	6
Quick Reference	7

Audience: Engineers who develop, test, and contribute to SystemC/TLM IP models and virtual-platform projects.

Last updated: 2026-05-25

Day-to-Day Workflow

Create a feature branch

Always branch from `main` (or the active release branch you are targeting). Never commit directly to `main`.

```
git checkout main
git pull origin main
git checkout -b feature/add-mailbox-test
```

Make changes and commit

Work in the appropriate area of the tree:

Change type	Typical location
New or updated IP model	<subsystem>/peripherals/<ip>/
IP documentation	<subsystem>/peripherals/<ip>/doc/
Subsystem architecture / test plan	doc/
Shared tooling	scripts/
VP integration / platform	<vp-repo>/vp/src/platform/
Firmware / DV test	<vp-repo>/sw/

```
# Stage your changes
git add smc/peripherals/clint/test/clint_tb.cpp
```

```
# Commit with a meaningful message
git commit -m "Add CLINT MTIMECMP boundary test"
```

Validates MTIP assertion when MTIME crosses MTIMECMP with 32-bit half-word access sequence."

Commit frequently — small, focused commits are easier to review and bisect. Each commit should leave the build in a working state for the components you touched.

Keep your branch up to date

Before pushing (and periodically during long-lived branches), rebase onto the latest target branch:

```
git fetch origin
git rebase origin/main
```

If you prefer merge over rebase:

```
git fetch origin
git merge origin/main
```

Rule of thumb: Use rebase for local unpushed work. If your branch is already pushed and shared with others, use merge to avoid rewriting shared history.

Push and create a Pull Request

```
# First push – sets upstream tracking
git push -u origin feature/add-mailbox-test
```

Then create a Pull Request on GitHub:

1. Go to your repository on GitHub
2. Click **Compare & pull request** (GitHub shows a banner after your push)
3. Set the base branch to `main` (or the agreed release branch)
4. Fill in the description — explain what changed and why
5. Request reviewers

6. Submit

After your PR merges

Clean up your local and remote branches:

```
git checkout main
git pull origin main
```

```
# Delete local feature branch
git branch -d feature/add-mailbox-test
```

```
# Delete remote feature branch (if GitHub didn't auto-delete it)
git push origin --delete feature/add-mailbox-test
```

Best Practices

Branching conventions

Prefix	Use for	Example
feature/	New functionality	feature/add-dma-burst-test
bugfix/	Bug fixes	bugfix/fix-plic-priority
refactor/	Code restructuring (no behavior change)	refactor/cleanup-register-api
ci/	CI/CD pipeline changes	ci/add-regression-stage
docs/	Documentation only	docs/update-developer-guide

- Keep branch names lowercase, hyphen-separated
- Include a brief description of the change
- Delete branches after merge

Commit messages

Follow this format:

<short summary – imperative mood, ≤72 chars>

<optional body – explain WHY, not just WHAT>

Good examples:

Add AXI4-Lite register read-back test for GPIO

Validates that all CSR registers return the correct reset values after a soft reset sequence.

Fix race condition in mailbox IRQ handler

The ISR was clearing the status bit before reading the payload, causing lost messages under back-to-back interrupt scenarios.

Bad examples:

Message	Problem
updated files	Too vague
fixed bug	Which bug?
WIP	Don't push WIP commits to main

What NOT to commit

Avoid	Why
Build artifacts (build/, *.o, *.so, *.elf)	Bloats repo; covered by .gitignore
IDE settings (.vscode/, .idea/)	Personal config; use global gitignore
Large binaries (firmware blobs, VDK images)	Use Git LFS or artifact storage
Credentials, tokens, passwords	Security risk — history is permanent
Generated files reproducible from source	Unnecessary churn
Waveform dumps (*.vcd, *.fsdb)	Large, ephemeral debug output

The root .gitignore should exclude common build and simulation artifacts. When adding a new model, verify its local build directory is covered.

Code review etiquette

As an author:

- Keep PRs small and focused — one logical change per PR
- Write a clear PR description with test evidence (build output, sim log, coverage delta)
- Respond to all review comments
- Don't force-push during active review without notifying reviewers

As a reviewer:

- Review within 1–2 business days
- Be constructive — suggest alternatives, not just “this is wrong”
- Approve when satisfied; don't block on style nits

IP model conventions

Every peripheral under <subsystem>/peripherals/<ip>/ must follow this layout:

```
<subsystem>/peripherals/<ip>/
├─ CMakeLists.txt           # Static library (e.g. smc_<ip>)
├─ README.md                # Interface summary, register map, build quick-start
├─ run_tests.sh              # Single-command build + test (Release / ASan /
coverage)
├─ include/                  # Public headers
├─ src/                      # Implementation
├─ test/                     # Self-checking testbench + CTest registration
└─ doc/                      # Numbered spec, LLD, test plan
    ├─ 01_<IP>_Specification.md
    ├─ 02_<IP>_LowLevel_Design.md
    ├─ 03_<IP>_Test_Plan.md
    └─ figures/
```

Shared file patterns:

Pattern	Purpose
CMakeLists.txt (root)	Declares the static library; links SystemC::systemc and SystemC::cci; exposes ENABLE_ASAN and ENABLE_COVERAGE options
CMakeLists.txt (test/)	Adds testbench executable(s), registers with CTest; scoped pass/fail regex properties
run_tests.sh	Build + test driver; auto-detects SYSTEMC_HOME and CCI_HOME; supports --asan, --coverage, --ctest, --clean
doc/build_docs.sh	Regenerates all IP documentation PDFs
doc/figures/_build_svgs.py	Programmatically generates block / pipeline / state-machine SVGs

Design rules:

- Use **CCI 1.0** (cci_param) for runtime configuration
 - Place shared TLM extensions in include/<subsystem>_tlm_extensions.h (one canonical copy per subsystem)
 - Name documentation files with numeric prefixes (01_, 02_, 03_) for consistent ordering
 - Trace specifications to the hardware spec and RTL register definitions where applicable
 - Add the IP to the subsystem's top-level CMakeLists.txt when integrating into a VP build
-

Working with Models

Standalone model development

Develop and test a model in isolation before integrating it into a VP:

```
cd smc/peripherals/clint
./run_tests.sh
```

Other useful modes:

```
./run_tests.sh --asan      # AddressSanitizer pass
./run_tests.sh --coverage  # Coverage report (LLVM on macOS, gcov on Linux)
./run_tests.sh --ctest     # Run via CTest with verbose output
```

Or build manually with CMake:

```
mkdir -p build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Debug
make
ctest --output-on-failure
```

Each IP's README.md documents the module interface, register map, and test groups.

VP-integrated development

After standalone tests pass, build the full platform in the VP repo:

```
cd riscv-vp-plusplus/vp/build
cmake .. -DVP_SYSC_BACKEND=accellera -DBUILD_TESTS=OFF
make sep-vp
```

Run subsystem-level tests:

```
cd ../../sw/sep-vp-tests/sep-gpio-test
make sim
```

Cross-model changes

When a change spans an IP model and VP integration (e.g., new register in smc/peripherals/plic/ plus address map update in the VP platform):

1. Implement and unit-test the IP first (./run_tests.sh)
2. Update the VP platform and runtime configuration (.ini / JSON)
3. Add or extend a subsystem test under <vp-repo>/sw/sep-vp-tests/
4. Submit a single PR if tightly coupled; otherwise merge the IP change first, then the integration PR

Documentation and PDF export

Regenerate IP documentation PDFs from the IP's doc/ directory:

```
cd smc/peripherals/clint/doc
./build_docs.sh
```

Or convert any Markdown file using the shared script:

`scripts/md-to-pdf.sh doc/01_SMC_Architecture.md`

`scripts/md-to-pdf.sh smc/peripherals/clint/doc/`

Quick Reference

Task	Command
Clone repository	<code>git clone git@github.com:tenstorrent/tt-oca-sim.git</code>
Create feature branch	<code>git checkout -b feature/<name></code>
Stage changes	<code>git add <files></code>
Commit	<code>git commit -m "<message>"</code>
Sync with main	<code>git fetch origin && git rebase origin/main</code>
Push (first time)	<code>git push -u origin feature/<name></code>
Push (subsequent)	<code>git push</code>
Delete merged branch (local)	<code>git branch -d feature/<name></code>
Delete merged branch (remote)	<code>git push origin --delete feature/<name></code>
Build + test IP	<code>cd <subsystem>/peripherals/<ip> && ./run_tests.sh</code>
IP coverage	<code>./run_tests.sh --coverage</code>
Export docs to PDF	<code>scripts/md-to-pdf.sh <file-or-dir></code>
Build VP	<code>cd riscv-vp-plusplus/vp/build && cmake .. -DVP_SYSC_BACKEND=accellera && make sep-vp</code>
Run subsystem sim	<code>cd riscv-vp-plusplus/sw/sep-vp-tests/<test> && make sim</code>
View log	<code>git log --oneline -20</code>
View remote URL	<code>git remote -v</code>
Check branch status	<code>git status</code>